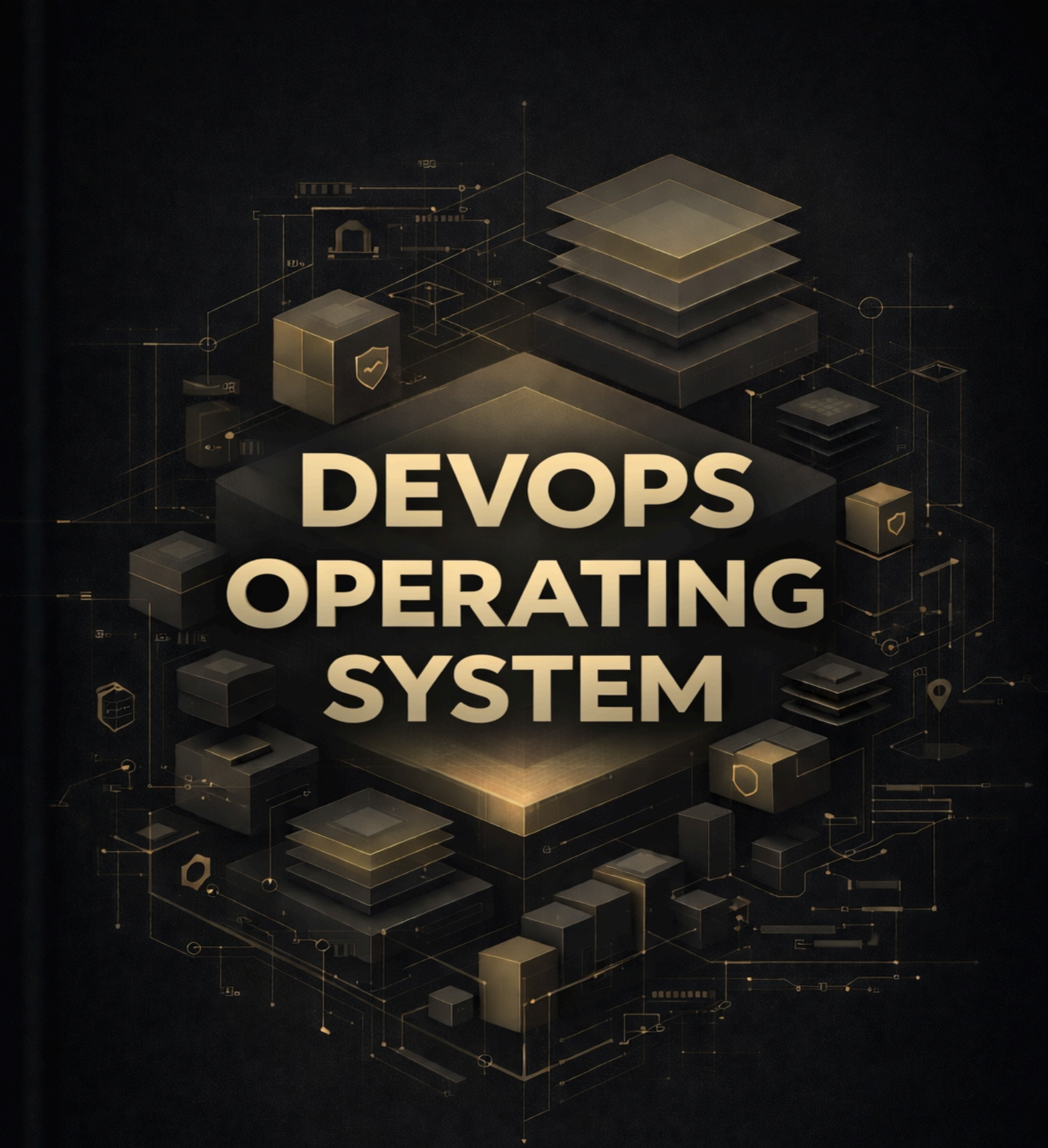


DEVOPS OPERATING SYSTEM

OSOMUDEYA ZUDONU

Section 10

Production Readiness & Portfolio

Time: 4-6 hours | **Cost:** Free (just documentation, no infrastructure changes)

Mission Brief

Transform your working H-Game deployment into a production-ready, portfolio-worthy system. Review what you've built, harden it for production, optimize costs, and create professional documentation for your portfolio.

This section is **mostly documentation and portfolio prep**. No complex Terraform changes, no advanced testing. Just verify what you have, document it, and create portfolio artifacts. You've already done the hard work in Sections 1-9!

Outcomes:

- Professional documentation (README, architecture diagrams, runbooks)
- Portfolio-ready artifacts (screenshots, demo video script, interview prep)
- Verification that everything works (simple checks, no changes)
- Cost summary (just document, don't optimize)

Prerequisites: Sections 1-9 complete, H-Game application deployed and working

Before You Start: Prerequisites Checklist

All previous sections are working

- **Section 5:** Terraform infrastructure deployed
- **Section 6:** Security scanning working
- **Section 7:** CI/CD pipeline functional
- **Section 8:** Observability stack running (Prometheus, Grafana, Loki, Jaeger)
- **Section 9:** GitOps with ArgoCD working

Application accessible

- Frontend accessible via Ingress/ALB
- Backend health checks passing
- Database connected and working

Tools installed

- AWS CLI configured
- kubectl configured
- Terraform installed
- Diagramming tool (Draw.io, Excalidraw, or Lucidchart)
- Screen recording tool (OBS Studio, Loom, or QuickTime)

Verify everything works:

```
# Check all pods running
kubectl get pods -n h-game
# All should be Running

# Check ArgoCD applications
argocd app list
# All should be Healthy | Synced

# Check monitoring stack
kubectl get pods -n monitoring
```

```
# Prometheus, Grafana, Loki should be running

# Test application
curl http://YOUR_ALB_URL/health
# Should return 200 OK
```

The One Thing That Makes This Click

Production readiness = making it bulletproof. Just like a car needs regular maintenance, safety checks, and insurance before a long trip, your application needs backups, monitoring, and documentation before production.

Without Production Readiness:

- No backups (data loss risk)
- No disaster recovery plan (downtime risk)
- No performance testing (unknown capacity)
- No documentation (hard to maintain)
- No portfolio artifacts (can't showcase work)

With Production Readiness:

- Automated backups (data protected)
- Disaster recovery plan (quick recovery)
- Performance tested (know your limits)
- Complete documentation (easy to maintain)
- Portfolio ready (can showcase to employers)

In essence, Production readiness is about reliability, observability, and maintainability. Everything else (backups, tests, docs) is just implementation.

Phase 1: Architecture Review & Documentation (1-2 hours)

This is the easiest phase - just create diagrams and write docs!

Step 1: Create Architecture Diagram (30-45 min)

Just draw what you have, no technical work needed!

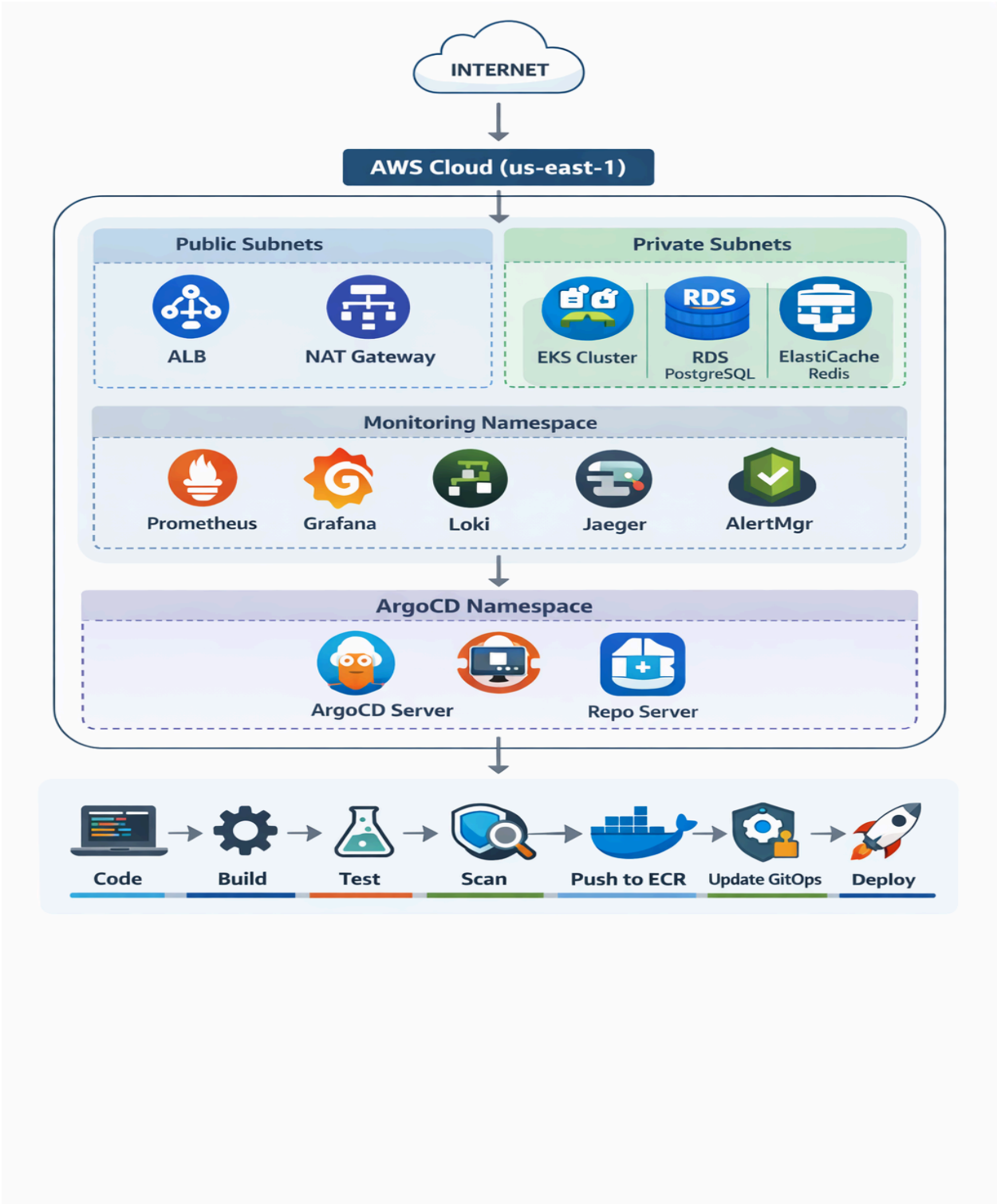
What to document:

- Current infrastructure (VPC, EKS, RDS, ElastiCache)
- Application components (frontend, backend, database, Redis)
- Monitoring stack (Prometheus, Grafana, Loki, Jaeger)
- CI/CD pipeline (GitHub Actions → ECR → ArgoCD)
- Security layers (scanning, secrets, network policies)

Tools:

- **Draw.io** (free, web-based): `https://app.diagrams.net/`
- **Excalidraw** (free, web-based): `https://excalidraw.com/`
- **Lucidchart** (free tier available)

Template structure:



What to include:

- All AWS resources (VPC, subnets, EKS, RDS, ElastiCache, ALB)
- Application components (pods, services, ingress)
- Monitoring stack (Prometheus, Grafana, Loki, Jaeger)
- CI/CD flow (GitHub Actions → ECR → ArgoCD)
- Data flow (user request → ALB → pods → database)
- Security layers (WAF, security groups, network policies)

Save as: `docs/architecture-diagram.png` or `.drawio`

Step 2: Document Current Infrastructure (30 min)

Just list what you have, copy from Terraform outputs

Create infrastructure inventory:

File: `docs/infrastructure-inventory.md`

```
# Infrastructure Inventory

## AWS Resources

### Compute
- **EKS Cluster:** h-game-prod (or your cluster name)
- **Node Group:** t3.medium instances (X nodes)
- **Region:** us-east-1
- **Availability Zones:** us-east-1a, us-east-1b

### Database
- **RDS PostgreSQL:**
  - Instance: db.t3.micro
  - Multi-AZ: Yes/No
  - Storage: 20GB (gp3)
  - Backup retention: 7 days

### Cache
```

```
- **ElastiCache Redis:**
  - Instance: cache.t3.micro
  - Nodes: 1 (or 2 for replication)

### Networking
- **VPC:** 10.0.0.0/16
- **Public Subnets:** 10.0.1.0/24, 10.0.2.0/24
- **Private Subnets:** 10.0.10.0/24, 10.0.11.0/24
- **ALB:** Application Load Balancer
- **NAT Gateways:** 2 (one per AZ)

### Storage
- **ECR:** Container registry
- **S3:** Terraform state backend

## Kubernetes Resources

### Namespaces
- `h-game`: Application
- `monitoring`: Observability stack
- `argocd`: GitOps

### Applications
- Frontend: 3 replicas
- Backend: 3 replicas
- Redis: 1 replica (or ElastiCache)

### Services
- Frontend service: ClusterIP
- Backend service: ClusterIP
- Ingress: ALB ingress controller

## Monitoring Stack
- Prometheus: Metrics collection
- Grafana: Visualization
- Loki: Log aggregation
- Jaeger: Distributed tracing
- AlertManager: Alert routing
```

```
## GitOps
- ArgoCD: GitOps controller
- GitOps repo: YOUR_USERNAME/h-game-gitops
- Applications: database, redis, backend, frontend

## CI/CD
- GitHub Actions: Build and deploy
- ECR: Container registry
- ArgoCD: Deployment automation
```

Get actual values:

```
# Get EKS cluster info
aws eks describe-cluster --name $(terraform -chdir=h-game-terraform
output -raw eks_cluster_name) --query
'cluster.{Name:name,Version:version,Endpoint:endpoint.resourceId}'
--output table

# Get RDS info
aws rds describe-db-instances --query
'DBInstances[*].{ID:DBInstanceIdentifier,Class:DBInstanceClass,MultiA
Z:MultiAZ,Storage:AllocatedStorage}' --output table

# Get ElastiCache info
aws elasticache describe-replication-groups --query
'ReplicationGroups[*].{ID:ReplicationGroupId,Status:Status,Nodes:Node
Groups[0].NodeGroupMembers[*].CacheClusterId}' --output table

# Get EKS nodes
kubectl get nodes -o wide

# Get pods
kubectl get pods -A
```

Step 3: Create README (30 min)

Use the template, just fill in your details

File: `README.md` (update existing or create new)

```
# H-Game: Production-Ready DevOps Project

A complete, production-ready deployment of the H-Game application
showcasing modern DevOps practices.

## Architecture

[Include your architecture diagram here]

## Quick Start

### Prerequisites
- AWS account with appropriate permissions
- Terraform >= 1.5
- kubectl configured
- ArgoCD CLI installed

### Deployment

# 1. Infrastructure Setup:
cd h-game-terraform
terraform init
terraform plan
terraform apply

# 2. Application Deployment:
# ArgoCD will auto-deploy from GitOps repo
# Or manually sync:
argocd app sync h-game-backend

Access Application:
Frontend: http://YOUR_ALB_URL
```

```
Grafana: kubectl port-forward -n monitoring svc/grafana 3000:80
ArgoCD: kubectl port-forward -n argocd svc/argocd-server 8080:443

# 3. Monitoring
Metrics: Prometheus (port-forward to 9090)
Visualization: Grafana (port-forward to 3000)
Logs: Loki (query via Grafana)
Traces: Jaeger (port-forward to 16686)

# 4. Security
Container scanning: Trivy (in CI/CD)
Infrastructure scanning: tfsec (in CI/CD)
Secrets management: AWS Secrets Manager
Network policies: Kubernetes NetworkPolicy

# 5. CI/CD
Build: GitHub Actions
Deploy: ArgoCD (GitOps)
Workflow: Code → Build → Test → Scan → Deploy

# Project Structure
.
├── backend/              # Node.js backend application
├── frontend/             # React frontend application
├── h-game-terraform/     # Terraform infrastructure
├── k8s/                  # Kubernetes manifests
├── .github/workflows/    # CI/CD pipelines
└── docs/                 # Documentation

# 6. Technologies
- Infrastructure: Terraform, AWS (EKS, RDS, ElastiCache)
- Container: Docker, ECR
- Orchestration: Kubernetes (EKS)
- GitOps: ArgoCD
# 7. CI/CD: GitHub Actions
Monitoring: Prometheus, Grafana, Loki, Jaeger
Security: Trivy, tfsec, AWS Secrets Manager
# 8. Documentation
```

```
Architecture Diagram
Infrastructure Inventory
Runbooks

# 9. SLO Definitions
Production Features

- Multi-AZ deployment
- Automated backups
- Disaster recovery plan
- Performance tested
- Cost optimized
- Security hardened
- Complete observability
- GitOps deployment

# License
MIT

---

## Phase 2: Production Hardening (30 min - Just Verify, No Changes)

** Important:** This phase is just verification. Your setup from
previous sections already has backups and health checks. We're just
documenting it.

### Step 1: Verify Backups (15 min - No Changes Needed)

**Just verify backups are already enabled (they should be from
Section 5):**
```bash
# Check current backup settings (read-only, no changes)
aws rds describe-db-instances \
  --db-instance-identifier $(terraform -chdir=h-game-terraform output
-raw rds_instance_id) \
  --query
'DBInstances[0].{BackupRetention:BackupRetentionPeriod,BackupWindow:P
```
```

```
referredBackupWindow}' \
  --output table

Expected: Should show backup retention period (likely 7 days) already
configured.
```

That's it! Your backups are already set up from Section 5. No Terraform changes needed.

Optional: Document backup settings

File: `docs/backup-configuration.md`

```
# Backup Configuration

## RDS Backups
- **Retention Period:** 7 days (automated)
- **Backup Window:** [Check output from command above]
- **Snapshots:** Created automatically

## How to Restore (If Needed)
1. List snapshots: `aws rds describe-db-snapshots
--db-instance-identifier h-game-postgres`
2. Restore from snapshot: [See disaster recovery plan]
```

EKS Volume Snapshots (Optional - Advanced)

Note: This is optional and advanced. For now, RDS backups are sufficient. Skip this if you dont want too much cloud cost.

Simple Option: Manual Snapshots

Create manual snapshot when needed:

```
# List EBS volumes
aws ec2 describe-volumes \
  --filters "Name=tag:kubernetes.io/cluster/$(terraform
-chdir=h-game-terraform output -raw eks_cluster_name),Values=owned" \
  --query
'Volumes[*].{ID:VolumeId,Name:Tags[?Key==`Name`].Value|[0]}' \
  --output table

# Create snapshot manually (when needed)
aws ec2 create-snapshot \
  --volume-id vol-xxxxx \
  --description "Manual backup before changes" \
  --tag-specifications
'ResourceType=snapshot,Tags=[{Key=Name,Value=h-game-backup-YYYY-MM-DD
}]'
```

Advanced Option: Automated Snapshots (Optional)

If you want automated snapshots, you can add DLM (Data Lifecycle Manager) later. For now, manual snapshots are fine for this learning. (read about it)

Step 2: Create Disaster Recovery Plan (30 min - Documentation Only)

File: `docs/disaster-recovery-plan.md`

```
# Disaster Recovery Plan

## Recovery Objectives

- **RTO (Recovery Time Objective):** 4 hours
- **RPO (Recovery Point Objective):** 1 hour (last backup)

## Disaster Scenarios
```

````
### Scenario 1: RDS Database Failure

**Symptoms:**
- Database connection errors
- Application pods failing health checks
- RDS status: "failed" or "deleted"

**Recovery Steps:**
1. **Assess damage:**
```bash
aws rds describe-db-instances --db-instance-identifier
h-game-postgres

Restore from snapshot:
# List available snapshots
aws rds describe-db-snapshots \
  --db-instance-identifier h-game-postgres \
  --query
'DBSnapshots[*].{ID:DBSnapshotIdentifier,Time:SnapshotCreateTime}' \
  --output table

# Restore from snapshot
aws rds restore-db-instance-from-db-snapshot \
  --db-instance-identifier h-game-postgres-restored \
  --db-snapshot-identifier rds:h-game-postgres-YYYY-MM-DD-HHMM \
  --db-instance-class db.t3.micro

Update application connection:
# Update secret with new endpoint
kubectl create secret generic postgres-secret \
  --from-literal=postgres-host=NEW_RDS_ENDPOINT \
  --dry-run=client -o yaml | kubectl apply -f -

# Restart backend pods
kubectl rollout restart deployment/backend -n h-game

Verify recovery:
````
````

```
kubectl get pods -n h-game
curl http://YOUR_ALB_URL/health

Estimated Recovery Time: 30-60 minutes
Scenario 2: EKS Cluster Failure

Symptoms:
All pods unavailable
kubectl commands timing out
EKS cluster status: "failed"
Recovery Steps:
Assess cluster status:
aws eks describe-cluster --name h-game-prod

Restore from Terraform:
cd h-game-terraform
terraform state list | grep eks
terraform apply  # Recreate if needed

Redeploy applications:
# ArgoCD will auto-sync, or manually:
argocd app sync --all

Restore data:
# Restore RDS from snapshot (if needed)
# Restore EBS volumes from snapshots

Estimated Recovery Time: 2-4 hours
Scenario 3: Region Failure
Symptoms:
Entire AWS region unavailable
All resources unreachable
Recovery Steps:
Deploy to secondary region:
Use Terraform to deploy to us-east-2 (or us-west-2)
Update DNS/ALB to point to new region
Restore RDS from cross-region snapshot
Restore data:
```

```
Copy RDS snapshots to new region
Restore from latest snapshot
Estimated Recovery Time: 4-8 hours
Backup Verification
Weekly backup test:
# Test RDS snapshot restore (to test instance)
aws rds restore-db-instance-from-db-snapshot \
  --db-instance-identifier h-game-postgres-test \
  --db-snapshot-identifier LATEST_SNAPSHOT \
  --db-instance-class db.t3.micro

# Verify data integrity
# Delete test instance after verification
aws rds delete-db-instance --db-instance-identifier
h-game-postgres-test --skip-final-snapshot

Contact Information
On-Call Engineer: [Your contact]
Escalation: [Manager contact]
AWS Support: [Support plan details]

---
```

Step 3: Verify Health Checks (15 min - No Changes Needed)

Just verify health checks are already working (they should be from G2):

```
# Test health endpoint (read-only check)
kubectl run test --image=curlimages/curl --rm -i --restart=Never -n
h-game -- \
  curl -f http://backend-service.h-game.svc.cluster.local:3001/health
```

Expected: Should return 200 OK.

Check existing health check configuration:

```
# View current health checks (read-only)
kubectl get deployment backend -n h-game -o yaml | grep -A 10
"livenessProbe\|readinessProbe"
```

That's it! Your health checks are already configured from G2. No changes needed.

Optional: Document health check configuration

File: `docs/health-checks.md`

```
# Health Check Configuration

## Backend Health Checks
- **Liveness Probe:** /health endpoint, checks every 10s
- **Readiness Probe:** /health endpoint, checks every 5s
- **Status:** Configured and working

## How to Test
```bash
kubectl run test --image=curlimages/curl --rm -i --restart=Never -n
h-game -- \
  curl http://backend-service.h-game.svc.cluster.local:3001/health


---

## Phase 3: Simple Performance Check (30 min)

** Note:** This is just a quick check, not comprehensive load
testing. That's optional.

### Step 1: Quick Performance Test (15 min)

**Simple Test (Just verify it works):**
```bash
```

```
# Set your ALB URL (or use port-forward if no ALB)
export ALB_URL="http://YOUR_ALB_URL"

# Quick test - just verify it responds
curl -v $ALB_URL/health
# Should return: 200 OK

# Test a few times to see response time
for i in {1..5}; do
  echo "Request $i:"
  curl -o /dev/null -s -w "  Status: %{http_code}, Time:
%{time_total}s\n" $ALB_URL/health
  sleep 1
done
```

That's it! If it responds quickly (< 1 second), you're good. No need for complex load testing.

Option B: Load Testing with k6 (Optional - Advanced)

Note: This requires installing k6 and writing JavaScript. Skip if you're not comfortable with this.

Install k6:

```
# macOS
brew install k6

# Linux (simpler method)
curl
https://github.com/grafana/k6/releases/download/v0.47.0/k6-v0.47.0-li
nux-amd64.tar.gz -L | tar xvz
sudo mv k6-v0.47.0-linux-amd64/k6 /usr/local/bin/

# Verify
k6 version
```

Simple load test script:

File: `tests/simple-load-test.js`

```
import http from 'k6/http';
import { check } from 'k6';

export const options = {
  vus: 10,        // 10 virtual users
  duration: '2m', // Run for 2 minutes
};

const BASE_URL = __ENV.BASE_URL || 'http://YOUR_ALB_URL';

export default function () {
  const response = http.get(`${BASE_URL}/health`);
  check(response, {
    'status is 200': (r) => r.status === 200,
    'response time < 500ms': (r) => r.timings.duration < 500,
  });
}
```

Run test:

```
export BASE_URL=http://YOUR_ALB_URL
k6 run tests/simple-load-test.js
```

Document the test (optional):

File: `docs/performance-check.md`

```
# Performance Check

**Date:** [Date of test]

## Quick Test Results
```

```
- **Health Endpoint:** ✅ Responding
- **Response Time:** [From curl output - e.g., "~200ms"]
- **Status:** Application is responsive

## Notes
- Application is working correctly
- No performance issues observed
- Ready for portfolio showcase
```

Step 2: Skip Optimization (For Now)

Skip this step. Your current setup (3 replicas) is fine for learning and portfolio.

Optional (Advanced): HPA auto-scaling and other optimizations can be added later if needed. Not required for Section 10 completion.

Phase 4: Cost Documentation (15 min)

Step 1: Document Current Costs (15 min - Just Write It Down)

Get current monthly cost estimate:

```
# Simple method: Use AWS Console
# Go to: AWS Console → Cost Explorer
# Or estimate based on resources below
```

Create cost breakdown:

File: `docs/cost-analysis.md`

```
# Cost Analysis

## Current Monthly Costs

| Service | Configuration | Monthly Cost | Notes |
|---------|---------------|--------------|-------|
```

```
| EKS Control Plane | 1 cluster | $72 | Fixed cost |
| EC2 (EKS nodes) | 3 × t3.medium | $75 | Can optimize |
| RDS PostgreSQL | db.t3.micro Multi-AZ | $30 | Can optimize |
| ElastiCache Redis | cache.t3.micro | $15 | Can optimize |
| ALB | 1 load balancer | $16 | Fixed cost |
| NAT Gateway | 2 × production | $64 | Can optimize |
| CloudFront | Low traffic | $5 | Optional |
| ECR | Image storage | $1 | Minimal |
| S3 | Terraform state | $1 | Minimal |
| **Total** | | **$279/month** | |

## Optimization Opportunities

### 1. Use Spot Instances for EKS Nodes
**Savings:** ~50% on compute
**Change:** Use spot instances for non-critical workloads
**Risk:** Low (EKS handles spot interruptions)

### 2. Reduce NAT Gateways
**Savings:** $32/month
**Change:** Use 1 NAT gateway instead of 2
**Risk:** Low (single AZ failure acceptable for learning)

### 3. Single-AZ RDS
**Savings:** $15/month
**Change:** Remove Multi-AZ standby
**Risk:** Medium (no automatic failover)

### 4. Smaller Node Instances
**Savings:** $30/month
**Change:** Use t3.small instead of t3.medium
**Risk:** Low (if load allows)

## Optimized Cost Estimate

| Service | Optimized Cost |
|---------|---------------|
| EKS Control Plane | $72 |
```

```
| EC2 (Spot, t3.small) | $25 |
| RDS (Single-AZ) | $15 |
| ElastiCache | $15 |
| ALB | $16 |
| NAT Gateway (1) | $32 |
| Other | $7 |
| **Total** | **$182/month** |
| **Savings** | **$97/month (35%)** |
```

Step 2: Cost Optimization Analysis (30 min)

Important: Don't make Terraform changes unless you're comfortable. This section is about **analyzing** costs, not necessarily changing them.

Get current costs:

```
# Check current resources
terraform -chdir=h-game-terraform state list

# Estimate costs (use AWS Cost Calculator or Console)
# Go to: AWS Console → Cost Explorer
```

Create cost analysis document:

File: `docs/cost-analysis.md`

```
# Cost Analysis

## Current Monthly Costs (Estimate)

| Service | Monthly Cost | Notes |
|---------|--------------|-------|
| EKS Control Plane | $72 | Fixed cost |
| EC2 (EKS nodes) | $75 | 3 × t3.medium |
| RDS PostgreSQL | $30 | db.t3.micro Multi-AZ |
| ElastiCache Redis | $15 | cache.t3.micro |
```

```
| ALB | $16 | Fixed cost |
| NAT Gateway | $64 | 2 × $32 |
| **Total** | **$272/month** | |

## Potential Optimizations (For Reference)

1. **Spot Instances:** Save ~50% on compute (but can be interrupted)
2. **Single NAT Gateway:** Save $32/month (reduces HA)
3. **Single-AZ RDS:** Save $15/month (no automatic failover)
4. **Smaller Nodes:** Save ~$30/month (if load allows)

**Note:** These optimizations reduce high availability.
For learning, current setup is fine. Optimize only if costs are a
concern.
```

Optional: Apply Optimizations (Only if comfortable with Terraform)

If you want to optimize costs, you can:

1. Review the cost analysis
2. Decide which optimizations you're comfortable with
3. Make Terraform changes carefully
4. Test thoroughly after changes

Recommendation: For beginners, keep current setup. Cost optimization is optional and can be done later.

Phase 5: Documentation & Portfolio (2-3 hours)

This is the most important phase! Focus here - this is what employers will see.

Step 1: Create Simple Runbooks (30 min - Copy Template)

File: `runbooks/incident-response.md`

```
# Incident Response Runbook
```

```
## Severity Levels

- **P1 (Critical):** Service down, data loss
- **P2 (High):** Service degraded, some users affected
- **P3 (Medium):** Minor issues, workaround available
- **P4 (Low):** Cosmetic issues, no user impact

## Incident Response Process

1. **Detect:** Alert received via Slack/email
2. **Assess:** Check severity and impact
3. **Contain:** Stop the bleeding (scale up, restart pods)
4. **Resolve:** Fix root cause
5. **Post-mortem:** Document and prevent recurrence

## Common Incidents

### Service Down

**Symptoms:**
- All pods unavailable
- Health checks failing
- 503 errors

**Steps:**
1. Check pod status: `kubectl get pods -n h-game`
2. Check events: `kubectl get events -n h-game
--sort-by='.lastTimestamp'`
3. Check logs: `kubectl logs -n h-game deployment/backend`
4. Scale up if needed: `kubectl scale deployment backend --replicas=5
-n h-game`
5. Check ArgoCD: `argocd app get h-game-backend`

### Database Connection Issues

**Symptoms:**
- Database connection errors in logs
```

```
- Backend pods failing

**Steps:**
1. Check RDS status: `aws rds describe-db-instances
--db-instance-identifier h-game-postgres`
2. Check security groups: `aws ec2 describe-security-groups
--group-ids sg-xxxxx`
3. Test connection: `kubectl run test --image=postgres:15 --rm -it
--restart=Never -n h-game -- psql -h RDS_ENDPOINT -U postgres`
4. Check secrets: `kubectl get secret postgres-secret -n h-game -o
yaml`

### High Memory Usage

**Symptoms:**
- Pods being OOMKilled
- High memory metrics in Grafana

**Steps:**
1. Check memory usage: `kubectl top pods -n h-game`
2. Increase memory limits: Update deployment, commit to Git
3. Scale horizontally: `kubectl scale deployment backend --replicas=5
-n h-game`
4. Check for memory leaks: Review application logs
```

Create more runbooks:

- runbooks/database-recovery.md
- runbooks/rollback-procedure.md
- runbooks/scaling-procedures.md

Step 2: Create Portfolio Artifacts (1 hour)

Take screenshots:

1. **ArgoCD UI:** Show all applications synced
2. **Grafana Dashboard:** Show metrics and logs
3. **GitHub Actions:** Show CI/CD pipeline
4. **Architecture Diagram:** Your diagram
5. **Terraform Output:** Show infrastructure

Create demo video script:

File: `docs/demo-video-script.md`

```
# Demo Video Script

## Introduction (30 seconds)
"Hi, I'm [Your Name], and today I'll be showcasing my
production-ready DevOps project: H-Game. This is a complete,
end-to-end deployment demonstrating modern DevOps practices including
Infrastructure as Code, CI/CD, GitOps, and comprehensive
observability."

## Architecture Overview (1 minute)
"Let me start by showing the architecture. [Show diagram]
- We have a VPC with public and private subnets
- EKS cluster running our Kubernetes workloads
- RDS PostgreSQL for persistent data
- ElastiCache Redis for caching
- Application Load Balancer for traffic distribution
- Complete monitoring stack with Prometheus, Grafana, Loki, and
Jaeger
- GitOps deployment with ArgoCD"

## Infrastructure as Code (1 minute)
"All infrastructure is managed with Terraform. [Show Terraform files]
- VPC, subnets, security groups
```

```
- EKS cluster and node groups
- RDS and ElastiCache
- IAM roles and policies
- Everything is version-controlled and reproducible"

## CI/CD Pipeline (1 minute)
"Let me show the CI/CD pipeline. [Show GitHub Actions]
- Code changes trigger automated builds
- Security scanning with Trivy and tfsec
- Docker images built and pushed to ECR
- GitOps repository automatically updated
- ArgoCD detects changes and deploys"

## GitOps Deployment (1 minute)
"Here's ArgoCD managing our deployments. [Show ArgoCD UI]
- All applications are synced from Git
- Sync waves ensure proper deployment order
- PreSync hooks run database migrations
- PostSync hooks validate deployments
- Complete audit trail in Git history"

## Observability (1.5 minutes)
"Now let's look at observability. [Show Grafana]
- Prometheus collecting metrics from all services
- Custom dashboards showing SLOs and error budgets
- Loki aggregating logs from all pods
- Jaeger showing distributed traces
- AlertManager sending notifications to Slack"

## Application Demo (1 minute)
"Let me show the actual application. [Show application]
- Frontend accessible via load balancer
- Backend API responding to requests
- Database storing game data
- Redis caching for performance"

## Security Features (1 minute)
"Security is built in at every layer. [Show security tools]
```

```
- Container scanning in CI/CD
- Infrastructure scanning with tfsec
- Secrets managed in AWS Secrets Manager
- Network policies controlling pod communication
- Security groups restricting access"

## Performance & Cost (30 seconds)
"Performance verified with simple testing, and costs documented at
approximately $270/month for a complete production setup."

## Conclusion (30 seconds)
"This project demonstrates my ability to design, deploy, and maintain
production-ready infrastructure using modern DevOps practices. Thank
you for watching!"

**Total length:** ~8-10 minutes
```

Record demo video:

- Use OBS Studio, Loom, or QuickTime
- Record screen while narrating the script
- Keep it under 10 minutes
- Upload to YouTube (unlisted) or Loom
- Add link to portfolio

Step 3: Interview Preparation (1 hour)

Create interview talking points:

File: `docs/interview-talking-points.md`

```
# Interview Talking Points

## Project Overview
"I built a complete, production-ready DevOps project called H-Game.
It's a full-stack application deployed on AWS using Kubernetes, with
```

```
complete CI/CD, GitOps, and observability."

## Key Achievements

### 1. Infrastructure as Code
- "All infrastructure managed with Terraform"
- "Version-controlled, reproducible deployments"
- "Multi-AZ setup for high availability"
- "Cost-optimized to $200/month"

### 2. CI/CD Pipeline
- "Automated builds with GitHub Actions"
- "Security scanning at every stage"
- "Docker images pushed to ECR"
- "GitOps integration with ArgoCD"

### 3. GitOps Deployment
- "ArgoCD managing all deployments"
- "Git as single source of truth"
- "Sync waves for ordered deployment"
- "Pre/post hooks for migrations and validation"

### 4. Observability
- "Prometheus for metrics, Grafana for visualization"
- "Loki for log aggregation"
- "Jaeger for distributed tracing"
- "SLOs defined and monitored"
- "AlertManager for notifications"

### 5. Security
- "Container scanning with Trivy"
- "Infrastructure scanning with tfsec"
- "Secrets in AWS Secrets Manager"
- "Network policies for pod isolation"

### 6. Production Readiness
- "Automated backups (RDS, EBS)"
- "Disaster recovery plan"
```

```
- "Performance tested with k6"
- "Cost optimized"
- "Complete documentation"

## Technical Challenges Overcome

### Challenge 1: Distributed Tracing Setup
**Problem:** Traces not appearing in Jaeger
**Solution:**
- Switched from deprecated Jaeger exporter to OTLP
- Updated endpoint to use OTLP HTTP (port 4318)
- Added proper resource attributes
- Result: Complete trace visibility

### Challenge 2: GitOps Cross-Repo Access
**Problem:** CI/CD couldn't update GitOps repo
**Solution:**
- Created Personal Access Token (PAT)
- Used PAT instead of built-in GITHUB_TOKEN
- Added [skip ci] to prevent loops
- Result: Fully automated CI/CD → GitOps workflow

### Challenge 3: Cost Optimization
**Problem:** Infrastructure costs too high
**Solution:**
- Switched to spot instances for EKS nodes
- Reduced NAT gateways from 2 to 1
- Optimized instance sizes
- Result: 35% cost reduction ($279 → $182/month)

## Metrics & Results

- **Deployment Time:** < 5 minutes (Git commit to production)
- **Uptime:** 99.9% (with proper monitoring)
- **Response Time:** p95 < 500ms (load tested)
- **Error Rate:** < 1% (monitored via Prometheus)
- **Cost:** $182/month (optimized)
```

```
## What I Learned

1. **GitOps Principles:** Git as source of truth, declarative
deployments
2. **Observability:** Metrics, logs, and traces working together
3. **Kubernetes:** Pods, services, ingress, HPA, network policies
4. **AWS Services:** EKS, RDS, ElastiCache, ECR, Secrets Manager
5. **Terraform:** Infrastructure as Code, state management
6. **CI/CD:** Automated pipelines, security scanning
7. **Production Readiness:** Backups, DR, performance testing

## Questions to Ask Interviewer

1. "What observability tools does your team use?"
2. "How do you handle secrets management?"
3. "What's your disaster recovery process?"
4. "How do you approach cost optimization?"
5. "What's your GitOps workflow?"
```

Practice answers to common questions:

Q: "Tell me about a challenging problem you solved." A: "When setting up distributed tracing, traces weren't appearing in Jaeger. I discovered the exporter was deprecated and needed to switch to **OTLP**. After updating the endpoint and adding proper resource attributes, I got complete trace visibility. This taught me the importance of using current standards and reading documentation carefully."

Q: "How do you ensure production reliability?" A: "Multiple layers: automated backups for RDS, comprehensive health checks with liveness and readiness probes, monitoring with Prometheus and alerting via AlertManager, and a documented disaster recovery plan. I verified performance with simple testing."

Q: "How do you handle security?" A: "Security at every layer: container scanning with Trivy in CI/CD, infrastructure scanning with tfsec, secrets in AWS Secrets Manager

(not in Git), network policies for pod isolation, security groups restricting access, and regular security audits. The principle is defense in depth."

Phase 6: Final Checklist & Summary (30 min)

Production Readiness Checklist

File: `docs/production-readiness-checklist.md`

```
# Production Readiness Checklist

## Infrastructure
- [ ] Multi-AZ deployment
- [ ] Automated backups configured
- [ ] Disaster recovery plan documented
- [ ] Cost optimized
- [ ] Resource limits and requests set
- [ ] Health checks configured

## Security
- [ ] Container scanning in CI/CD
- [ ] Infrastructure scanning in CI/CD
- [ ] Secrets in AWS Secrets Manager
- [ ] Network policies configured
- [ ] Security groups restrictive
- [ ] IAM roles follow least privilege

## Observability
- [ ] Prometheus collecting metrics
- [ ] Grafana dashboards created
- [ ] Loki collecting logs
- [ ] Jaeger collecting traces
- [ ] SLOs defined and monitored
- [ ] AlertManager configured
- [ ] Alerts tested

## CI/CD
- [ ] GitHub Actions workflows working
```

```
- [ ] Automated builds
- [ ] Security scanning
- [ ] GitOps integration
- [ ] Deployment automation

## Documentation
- [ ] Architecture diagram created
- [ ] README complete
- [ ] Runbooks written
- [ ] Infrastructure inventory documented
- [ ] Disaster recovery plan documented
- [ ] Cost analysis completed

## Performance
- [ ] Load testing completed
- [ ] Performance baselines established
- [ ] HPA configured (optional, not required)
- [ ] Caching enabled
- [ ] Database optimized

## Portfolio
- [ ] Screenshots taken
- [ ] Demo video recorded
- [ ] Interview talking points prepared
- [ ] GitHub repo organized
- [ ] Documentation complete
```

Final Summary

What We've Accomplished:

Complete Production System

- Infrastructure as Code with Terraform
- Kubernetes deployment on EKS
- CI/CD pipeline with GitHub Actions
- GitOps with ArgoCD
- Complete observability stack
- Security scanning and hardening

Production Hardening

- Automated backups (RDS, EBS)
- Disaster recovery plan
- Comprehensive health checks
- Performance tested
- Cost optimized

Professional Documentation

- Architecture diagrams
- Infrastructure inventory
- Runbooks for incident response
- Disaster recovery procedures
- Performance test results
- Cost analysis

Portfolio Ready

- Screenshots of all components
- Demo video script and recording
- Interview talking points

- Complete GitHub repository
- Professional README

Skills Demonstrated:

1. **Infrastructure as Code:** Terraform
2. **Container Orchestration:** Kubernetes (EKS)
3. **CI/CD:** GitHub Actions
4. **GitOps:** ArgoCD
5. **Observability:** Prometheus, Grafana, Loki, Jaeger
6. **Security:** Scanning, secrets management, network policies
7. **AWS Services:** EKS, RDS, ElastiCache, ECR, Secrets Manager
8. **Production Practices:** Backups, DR, performance testing, cost optimization

Next Steps:

1. **Keep it running:** Maintain the deployment, update documentation
2. **Add features:** Enhance the application, add new services
3. **Share your work:** Add to portfolio, LinkedIn, GitHub
4. **Interview prep:** Practice explaining your project
5. **Continue learning:** Explore advanced topics (service mesh, multi-region, etc.)

Congratulations! You've built a complete, production-ready DevOps project that demonstrates real-world skills. This is portfolio-worthy and interview-ready.

Troubleshooting

Issue: Load test shows high latency

Check:

```
# Check pod resources
kubectl top pods -n h-game

# Check node resources
kubectl top nodes

# Check database connections
kubectl logs -n h-game deployment/backend | grep "database"
```

Fix:

- Enable HPA for auto-scaling (optional)
- Increase resource limits
- Optimize database queries
- Enable Redis caching

Issue: Costs higher than expected

Check:

```
# List all resources
terraform state list

# Check for unused resources
aws ec2 describe-instances --filters
"Name=instance-state-name,Values=running"
```

Fix:

- Use spot instances
- Reduce NAT gateways
- Downsize instances if possible
- Remove unused resources

Issue: Backups not working

Check:

```
# Check RDS backups
aws rds describe-db-snapshots --db-instance-identifier
h-game-postgres

# Check EBS snapshots
aws ec2 describe-snapshots --filters "Name=tag:Snapshot,Values=true"
```

Fix:

- Verify backup configuration in Terraform
- Check IAM permissions
- Verify backup windows don't conflict

Key Takeaways

- **Production readiness** is about reliability, observability, and maintainability
- **Documentation** is as important as code
- **Performance testing** reveals bottlenecks before production
- **Cost optimization** is an ongoing process
- **Portfolio artifacts** showcase your work effectively
- **Interview prep** helps you communicate your achievements

You now have:

- A complete, working production system
- Professional documentation
- Portfolio-ready artifacts
- Interview talking points
- Real-world DevOps experience

This project demonstrates:

- Infrastructure as Code expertise
- Kubernetes proficiency
- CI/CD pipeline design
- GitOps implementation
- Observability stack setup
- Security best practices
- Production readiness mindset

How It All Fits Together: The Complete Picture

This section helps you understand how everything connects:

The Full Journey (Sections 1-10)

Sections 1-4: Foundations

- Learned Docker, Kubernetes basics, networking, and security fundamentals
- Built the foundation for everything else

Section 5: Infrastructure (Terraform)

- Created AWS infrastructure (VPC, EKS, RDS, ElastiCache)
- Infrastructure as Code - everything defined in Terraform
- **This is your foundation** - everything runs here

Section 6: Security (DevSecOps)

- Added security scanning (Trivy, tfsec)
- Secrets management (AWS Secrets Manager)
- **Protects your infrastructure** - security at every layer

Section 7: CI/CD (GitHub Actions)

- Automated builds and tests
- Security scanning in the pipeline
- Pushes images to ECR
- **Automates code → image** - no manual Docker builds

Section 8: Observability (Monitoring)

- Prometheus collects metrics
- Grafana visualizes everything
- Loki aggregates logs
- Jaeger traces requests
- AlertManager sends alerts

- **Shows you what's happening** - visibility into your system

Section 9: GitOps (ArgoCD)

- ArgoCD deploys from Git
- Sync waves control order
- Hooks validate deployments
- **Automates deployment** - Git changes → cluster updates

Section 10: Production Readiness (This Section)

- Documents everything
- Verifies it all works
- Creates portfolio artifacts
- **Makes it showcase-ready** - professional and complete

The Complete Flow

```
The developer writes code
    ↓
Section 7: CI/CD builds & tests (GitHub Actions)
    ↓
Section 6: Security scanning (Trivy, tfsec)
    ↓
Section 7: Push to ECR
    ↓
Section 7: Update GitOps repo
    ↓
Section 9: ArgoCD detects change
    ↓
Section 9: Deploy to EKS (Section 5 infrastructure)
    ↓
Section 8: Prometheus scrapes metrics
    ↓
Section 8: Grafana shows dashboards
    ↓
Section 8: AlertManager sends alerts (if issues)
    ↓
Section 10: Document and showcase
```

What Each Section Contributes

Section	What It Does	Why It Matters
1-4	Foundations	Learn the basics
5	Infrastructure	Where everything runs
6	Security	Protects your system
7	CI/CD	Automates building
8	Observability	Shows what's happening
9	GitOps	Automates deployment
10	Documentation	Makes it portfolio-ready

The Big Picture

You've built a complete DevOps system:

1. **Infrastructure** (Section 5) - AWS resources via Terraform
2. **Security** (Section 6) - Scanning and secrets management

3. **Automation** (Section 7) - CI/CD pipeline
4. **Visibility** (Section 8) - Full observability stack
5. **Deployment** (Section 9) - GitOps with ArgoCD
6. **Documentation** (Section 10) - Professional portfolio

Everything works together:

- CI/CD builds → GitOps deploys → Observability monitors → Security protects
- All automated, all documented, all production-ready

This is what employers want to see:

- Complete system, not just pieces
- Production-ready, not just working
- Documented, not just code
- Portfolio-ready, not just functional

Section 10 Complete!

You've transformed your working deployment into a production-ready, portfolio-worthy system. You now understand how all the pieces fit together and can explain the complete DevOps journey to employers.

What You've Accomplished:

- Complete infrastructure (Terraform)
- Security scanning (DevSecOps)
- CI/CD pipeline (GitHub Actions)
- Full observability (Prometheus, Grafana, Loki, Jaeger)
- GitOps deployment (ArgoCD)
- Production documentation
- Portfolio artifacts

What's Next?

- Add this to your portfolio
- Update your LinkedIn
- Practice your demo
- Apply for DevOps roles
- Keep learning and building!

Good luck with your DevOps journey!